CVM++

A Stack-Based Virtual Machine in C++17

Technical Design & Implementation Report

Full pipeline: Lexer → Parser → Compiler → VM

Language: C++17 Build: CMake 3.10+ / g++ Platform: Cross-platform

May 2026

Inspired by *Crafting Interpreters* by Robert Nystrom

Contents

1 Project Overview

CVM++ is a complete, from-scratch implementation of a miniature programming language and its execution engine, written in C++17. It demonstrates the full compilation pipeline that real-world language runtimes follow: source text is first broken into tokens by a **Lexer**, those tokens are structured into an **Abstract Syntax Tree (AST)** by a **Parser**, the tree is translated into flat **bytecode** by a **Compiler**, and finally that bytecode is executed by a **stack-based Virtual Machine**.

The project did not arrive at its current state all at once. It began as a minimal integer-and-boolean calculator with a handful of opcodes, and grew incrementally: strings were added to the value system, then a full operator set, then user-defined functions with a call stack, and finally control-flow constructs such as `for` loops and `else if` chains. Each extension touched exactly the same four-layer pipeline in the same order — lexer, parser, compiler, VM — demonstrating that the architecture was correctly separated from the start.

The final codebase is approximately 1 500 lines of well-commented C++17 spread across nine source files.

1.1 Goals and Scope

The design priorities of CVM++ are:

- **Educational clarity** — every stage of the pipeline is isolated into its own class with a single, well-defined responsibility.
- **Correctness over performance** — the VM uses a simple sequential dispatch loop rather than computed gotos or JIT compilation.
- **Minimal dependencies** — the entire project uses only the C++17 standard library; no third-party frameworks are required.
- **Extensibility** — the architecture is intentionally layered so that new language features can be added without restructuring existing components, as demonstrated by the extensions described throughout this report.

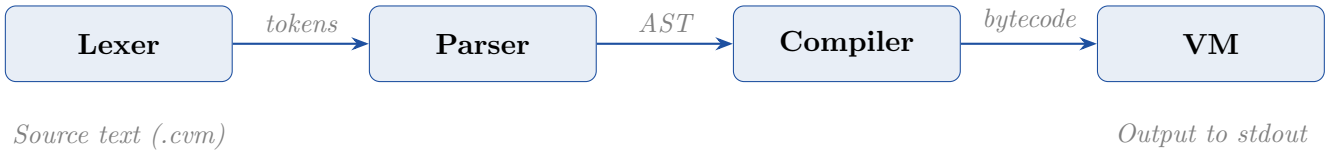
1.2 Supported Language Features

CVM++ began as a simple integer-based language and was gradually extended with strings, additional operators, user-defined functions, recursion, enhanced control-flow constructs, and richer input/output support, making it a substantially richer programming environment. The table below summarises the complete feature set available in the final version of the language.

Feature	Syntax / Example
Integer literals	`0`, `42`
Boolean literals	`true`, `false`
String literals	`"cat"`
Variable declaration	`let x = 10;`
Variable assignment	`x = x + 1;`
Arithmetic + - * /	`a + b * c`
Modulo	`n % 3`
Unary negation	`-x`
String concat	`"Hi, " + name`
Comparisons	`==`, `!=`, `<`, `>`, `<=`, `>=`
Logical operators	`&& || !`
Print statement	`print x;`
Integer input	`let n = input;`
String input	`let s = sinput;`
Conditional	`if / else if / else`
While loop	`while (cond) { }`
For loop	`for (init; cond; update) { }`
Functions	`fn add(a,b) { return a+b; }`
Recursion	`factorial(n-1)`
Single-line comment	`// comment`

2 System Architecture

The compilation pipeline follows four sequential, stateless stages. Each stage consumes the output of the previous one and produces a new data structure.



This architecture proved its worth throughout the project. Every new feature — strings, modulo, functions, for loops — followed exactly the same path through all four layers without requiring changes to unrelated components.

2.1 File Structure

File	Responsibility
src/token.h	TokenType enum and Token struct
src/lexer.h/.cpp	Lexer: source text → vector<Token>
src/ast.h	Full AST node hierarchy (Expr/Stmt subclasses)
src/parser.h/.cpp	Recursive-descent parser: tokens → AST
src/compiler.h/.cpp	AST walker: emits bytecode into a Chunk
src/opcode.h	OpCode enum shared between compiler and VM
src/vm.h/.cpp	Stack-based bytecode interpreter with call stack
src/main.cpp	Entry point: file runner and interactive REPL
CMakeLists.txt	CMake build configuration

3 Stage 1: Lexer

3.1 Responsibility

The Lexer class (lexer.h/.cpp) takes the raw source string and produces a flat vector<Token>. A Token is a struct holding three fields: a TokenType enum value, the raw text (value), and the source line number for error reporting.

3.2 Token Types

Initially, the lexer recognised a small set of tokens sufficient for integer arithmetic and basic control flow. As the language grew, new token types were introduced in layers. The table below shows the entire list of tokens after all the extensions.

Category	Token types
Literals	INT_LIT, STRING_LIT, TRUE_KW, FALSE_KW
Keywords	LET, IF, ELSE, FOR, WHILE, PRINT
	INPUT, SINPUT, FN, RETURN
Identifiers	IDENT
Arithmetic	PLUS, MINUS, STAR, SLASH, PERCENT
Comparison	ASSIGN, EQ_EQ, BANG_EQ, LESS, GREATER, LESS_EQ, GREATER_EQ
Logical	BANG, AND, OR
Delimiters	LPAREN, RPAREN, LBRACE, RBRACE, SEMICOLON, COMMA
Control	EOF_T

3.3 Implementation Details

The lexer maintains a position cursor pos and a line counter. It iterates character-by-character through the source string:

- **Whitespace and comments** are consumed by skipWhitespaceAndComments(), which handles spaces, tabs, carriage returns, newlines, and // line comments.
- **Numbers** are read by readNumber(), which advances while isdigit() is true and constructs an INT_LIT token.

- **Identifiers and keywords** are read by `readIdentOrKeyword()`, which scans alphanumeric or underscore characters and performs a lookup against a static `unordered_map` keyword table. New keywords (`fn`, `return`, `for`, `sinput`) were added by simply inserting entries into this map.
- **String literals** were added by detecting the opening `"` character before the main switch statement, scanning until the closing `"`, and tracking newlines inside the string to keep line numbers accurate. Unterminated strings raise a lexer error.
- **Single- and two-character tokens** are handled by a `switch` statement with one lookahead character. The initial implementation handled only `=` vs `==`; the extension added `!=`, `<=`, `>=`, `&&`, and `||` using the same lookahead pattern.

4 Stage 2: Parser

4.1 Responsibility

The `Parser` class (`parser.h/.cpp`) implements a **recursive-descent parser** that converts the flat token list into a typed Abstract Syntax Tree rooted at a `Program` node.

4.2 Grammar

The grammar grew alongside the language. The initial grammar handled only integer arithmetic and simple control flow. The final grammar is shown below, with extensions clearly visible as new rules and new alternatives.

```
program    -> fnDecl* stmt* EOF
fnDecl     -> "fn" IDENT "(" params ")" block
params     -> IDENT ("," IDENT)*
stmt       -> letStmt | assignStmt | ifStmt | whileStmt
           | forStmt | printStmt | returnStmt
letStmt    -> "let" IDENT "=" expr ";"
assignStmt -> IDENT "=" expr ";"
ifStmt     -> "if" "(" expr ")" block
              ("else" ("if" ... | block))?
whileStmt  -> "while" "(" expr ")" block
forStmt    -> "for" "(" init ";" expr ";" update ")" block
returnStmt -> "return" expr? ";"
printStmt  -> "print" expr ";"
block      -> "{" stmt* "}"
expr       -> logical
logical    -> comparison (("&&" | "||") comparison)*
comparison -> addition (("==" | "!=" | "<" | ">"
                         | "<=" | ">=") addition)*
addition   -> multiply (("+" | "-") multiply)*
multiply   -> primary (("*" | "/" | "%") primary)*
primary    -> INT_LIT | STRING_LIT | "true" | "false"
           | IDENT | IDENT "(" args ")" | "input"
```

```
| "sinput" | "-" primary | "!" primary
| "(" expr ")"
```

Operator precedence is baked into the grammar hierarchy. The `parseLogical()` layer was inserted between `parseExpr()` and `parseComparison()` when `&&` and `||` were added, giving logical operators the lowest precedence above assignment — exactly as they behave in C and Python.

4.3 AST Node Hierarchy

The AST is defined entirely in `ast.h` using virtual base classes and `unique_ptr` ownership. The table below shows the complete final set of nodes.

Node Type	Represents
*Expressions (subclass of **`Expr`**)*	
`IntLitExpr`	Integer constant
`BoolLitExpr`	Boolean constant
`StringLitExpr`	String literal
`VarExpr`	Variable read
`BinaryExpr`	Binary operation (op char + two children)
`UnaryExpr`	Unary `-` or `!`
`AndExpr`	Short-circuit `&&`
`OrExpr`	Short-circuit `||`
`InputExpr`	Read integer from stdin
`SInputExpr`	Read string from stdin
`CallExpr`	Function call with argument list
*Statements (subclass of **`Stmt`**)*	
`LetStmt`	Variable declaration
`AssignStmt`	Variable re-assignment
`PrintStmt`	Print expression
`IfStmt`	Conditional with optional else
`WhileStmt`	While loop
`ForStmt`	For loop (compiled as while sugar)
`FnDecl`	Function declaration
`ReturnStmt`	Return from function
`Program`	Root node

4.4 Short-Circuit Evaluation for `&&` and `||`

Logical `&&` and `||` are not represented as `BinaryExpr` nodes. They require **short-circuit evaluation** — the right operand must not be evaluated if the left operand already determines the result. Representing them as ordinary binary nodes would require the compiler to evaluate both sides unconditionally. Instead, dedicated `AndExpr` and `OrExpr` nodes signal to the compiler that jump-based short-circuit code must be emitted.

4.5 Error Reporting

Every `expect()` call on a mismatched token throws a `std::runtime_error` that includes the line number and the offending token text, e.g.: *"Parse error at line 5: Expected ';' after let statement (got 'while')"*.

5 Stage 3: Compiler

5.1 Responsibility

The `Compiler` class (`compiler.h/.cpp`) performs a single-pass walk of the AST and emits a flat byte array (a `Chunk`) that encodes the program as bytecode instructions for the VM.

5.2 The Chunk Structure

A `Chunk` is a thin wrapper around `vector<uint8_t>` with helper methods for emission and backpatching. When strings were added, a `stringPool` (`vector<string>`) was appended to the `Chunk` so that string constants could be stored once and referenced by index in the bytecode, rather than being inlined as raw bytes.

- `emit(byte/op)` — append a single byte or opcode.
- `emitInt32(v)` — append a 32-bit integer in big-endian order.
- `emitJump(op)` — emit a jump opcode followed by a two-byte placeholder `0xFFFF`.
- `patchJump(pos, target)` — overwrite the placeholder with the real target address (backpatching).
- `addString(s)` — add a string to the pool and return its index.

5.3 Variable Management

Variables are stored in a flat array of 256 slots per call frame. The compiler maintains an `unordered_map<string, uint8_t>` that maps variable names to slot indices. Initially this was a single global map. When functions were added, the map was saved and restored around each function compilation, giving each function its own private variable namespace.

5.4 Control Flow and Backpatching

Because the target address of a jump is not known when the jump instruction is first emitted, a two-phase approach is used:

1. **Emit a placeholder**: `emitJump(JUMP_IF_FALSE)` writes the opcode and two `0xFF` bytes. The position of those bytes is saved.
2. **Compile the body** of the branch or loop.

3. **Patch the placeholder**: `patchJump(saved_pos, current().here())` overwrites the two bytes with the now-known target address.

The `for` loop is compiled as syntactic sugar over `while`: the init statement is emitted first, then a standard condition-check-and-jump, then the body, then the update statement, then an unconditional jump back to the condition. No new opcodes were required.

5.5 Function Compilation and the `current()` Pattern

When functions were added, the compiler faced a new challenge: it needed to emit bytecode into two different destinations — the main chunk and the function's own chunk — depending on what was being compiled. This was solved cleanly with a `current()` helper method:

Listing 1: current() routes emissions to the right Chunk

```
FunctionObject* currentFn = nullptr;

Chunk& current() {
    return currentFn ? currentFn->chunk : chunk;
}
```

Every emit call in `compileStmt()` and `compileExpr()` uses `current()` rather than `chunk` directly. When `compileFn()` sets `currentFn` to point at the function being compiled, all emissions automatically go to the correct destination. When it resets `currentFn` to `nullptr`, emissions return to the main chunk. This required no structural changes to the statement or expression compilers.

5.6 Short-Circuit Compilation

`AndExpr` and `OrExpr` are compiled using jumps rather than opcodes. For `&&`: if the left operand is falsy, a `JUMP` skips the right operand and pushes `false`. For `||`: if the left operand is truthy, a `JUMP` skips the right operand and pushes `true`. This mirrors how C compilers implement short-circuit evaluation.

6 Stage 4: Virtual Machine

6.1 Architecture

The VM (`vm.h/.cpp`) is a **stack-based interpreter**. All operations work on an implicit operand stack (`vector<Value>`). There are no registers.

Initially the VM held a single instruction pointer `ip` and a flat `vars[256]` array. When functions were added, both were promoted into a `CallFrame` struct, and a vector of frames replaced the single global state. The stack itself remained shared across frames, with each frame recording its `stackBase` offset.

6.2 Value Representation

The `Value` struct evolved over the course of the project. Initially it was a tagged union holding either a 32-bit integer or a boolean:

Listing 2: Original Value struct (integers and booleans only)

```
enum class ValType { INT, BOOL };

struct Value {
    ValType type = ValType::INT;
    union { int32_t i = 0; bool b; };
};
```

When strings were added, the union had to be broken. `std::string` has a constructor and destructor, which the C++ standard forbids inside a union without explicit lifetime management. The fix was straightforward — promote all members to plain struct fields:

Listing 3: Extended Value struct (integers, booleans, strings)

```
enum class ValType { INT, BOOL, STRING };

struct Value {
    ValType type = ValType::INT;
    int32_t     i = 0;
    bool        b = false;
    std::string s;

    bool truthy() const {
        if (type == ValType::BOOL)   return b;
        if (type == ValType::STRING) return !s.empty();
        return i != 0;
    }
};
```

The `truthy()` method was extended to handle the three types: booleans use their own value, strings are truthy when non-empty, and integers are truthy when non-zero. This allowed `!` and `JUMP_IF_FALSE` to work uniformly across all types without special cases in each opcode.

6.3 The Call Stack

Functions required the most significant structural change to the VM. The single global `ip` and `vars` were replaced by a `vector<CallFrame>`, where each frame owns its own instruction pointer, variable slots, and a pointer to the bytecode chunk being executed:

Listing 4: CallFrame struct

```
struct CallFrame {
    const Chunk*        chunk;      // which chunk we're
        executing
    size_t              ip;         // instruction pointer for
        this frame
    size_t              stackBase;  // unused stack slots
        below this frame
    std::vector<Value> vars;        // 256 variable slots for
        this call
```

```
};
```

On `CALL`, the VM looks up the function by name in the function table, checks the argument count, copies arguments into the new frame's variable slots, and pushes the frame. On `RETURN`, the frame is popped and the return value is pushed onto the stack for the caller to consume. This design supports arbitrarily deep recursion limited only by available memory.

6.4 Instruction Set

The complete instruction set in the final version is listed below.

Opcode	Operand bytes	Effect
PUSH_INT	[b3] [b2] [b1] [b0]	Push 32-bit int (big-endian)
PUSH_TRUE	---	Push boolean `true`
PUSH_FALSE	---	Push boolean `false`
PUSH_STR	[idx]	Push string from pool[idx]
LOAD	[idx]	Push `vars[idx]`
STORE	[idx]	Pop → `vars[idx]`
ADD	---	Pop b, a; push $a + b$ or concat
SUB	---	Pop b, a; push $a - b$
MUL	---	Pop b, a; push $a \times b$
DIV	---	Pop b, a; push a/b (int)
MOD	---	Pop b, a; push $a \bmod b$
NEG	---	Pop a; push $-a$
NOT	---	Pop a; push `!truthy(`a`)`
EQ	---	Pop b, a; push $a == b$
NEQ	---	Pop b, a; push $a \neq b$
LESS	---	Pop b, a; push $a < b$
GREATER	---	Pop b, a; push $a > b$
LESS_EQ	---	Pop b, a; push $a \leq b$
GREATER_EQ	---	Pop b, a; push $a \geq b$
PRINT	---	Pop and print to stdout
INPUT	---	Read int from stdin, push
SINPUT	---	Read string from stdin, push
JUMP	[hi] [lo]	`ip` ← address
JUMP_IF_FALSE	[hi] [lo]	Pop; if falsy → `ip` ← address
CALL	[argc] [nameIdx]	Set up new call frame
RETURN	---	Tear down frame, push return value
POP	---	Discard top of stack
HALT	---	Stop execution

A notable design decision: `ADD` was extended to handle string concatenation at runtime by checking the operand types before choosing between integer addition and string concatenation. This avoided adding a separate `CONCAT` opcode and kept the compiler simple — it always emits `ADD` for `+`, and the VM resolves the correct behaviour dynamically.

6.5 Dispatch Loop

The VM runs nested loops: an outer loop over the frame stack, and an inner `while` loop that dispatches one opcode per iteration. When `CALL` or `RETURN` changes the active frame, the inner loop sets a `halt` flag to break out and re-enter the outer loop with the updated frame reference. Division and modulo by zero, stack underflow, type mismatches on comparison operators, and unknown opcodes all raise `std::runtime_error`.

7 Entry Point and REPL

`main.cpp` supports two modes of operation:

- **File mode** (`./cvm script.cvm`): reads the entire file into a string and passes it through the four-stage pipeline via `runSource()`.
- **REPL mode** (`./cvm`): displays a welcome banner and repeatedly prompts with `>>`, running each line through the full pipeline independently. The loop exits on `exit`, `quit`, or EOF.

The selection is entirely based on `argc`: if `argc == 2` a filename was provided; otherwise the REPL starts. Both paths converge on the same `runSource()` function, which runs all four pipeline stages and catches any `std::exception`, printing it in red to `stderr` without crashing.

8 Example Programs

Four example programs are included in the `examples/` directory. Together they exercise every major feature of the language.

8.1 Calculator

A user-facing integer calculator demonstrating functions, integer input, arithmetic, and a full `if / else if / else` chain. Each arithmetic operation is wrapped in its own function, making the dispatch logic clean and readable.

Listing 5: calculator.cvm (condensed)

```
fn add(a, b)      { return a + b; }
fn subtract(a, b) { return a - b; }
fn multiply(a, b) { return a * b; }
fn divide(a, b)   { return a / b; }
fn modulo(a, b)   { return a % b; }

print "Please enter the first number:";
let a = input;
print "Please enter the second number:";
let b = input;
print "1=Add 2=Sub 3=Mul 4=Div 5=Mod";
let op = input;
```

```
13
14 if (op == 1) { print add(a, b); }
15 else if (op == 2) { print subtract(a, b); }
16 // ... and so on
```

8.2 Prime Number Checker

Prints all prime numbers from 2 to 50 using an `isPrime` function with a `while` loop, and a `for` loop in the driver. This example showcases modulo, the `for` loop, early `return` from a function, and the `<=` comparison operator — none of which existed in the initial version of CVM++.

Listing 6: primes.cvm

```
1  fn isPrime(n) {
2      if (n < 2) { return false; }
3      let i = 2;
4      while (i * i <= n) {
5          if (n % i == 0) { return false; }
6          i = i + 1;
7      }
8      return true;
9  }
10
11 for (let i = 2; i <= 50; i = i + 1) {
12     if (isPrime(i)) { print i; }
13 }
```

8.3 String Operations and Grading

Demonstrates `sinput`, string concatenation, functions that return strings, and a multi-branch `else if` chain. The `classify` function maps a numeric score to a letter grade, showing that strings can be first-class return values.

Listing 7: strings.cvm

```
1  fn greet(name) { return "Hello, " + name + "!"; }
2
3  fn classify(score) {
4      if (score >= 90) { return "A"; }
5      else if (score >= 80) { return "B"; }
6      else if (score >= 70) { return "C"; }
7      else if (score >= 60) { return "D"; }
8      else { return "F"; }
9  }
10
11 let name = sinput;
12 print greet(name) + " Please enter your score:";
13 let score = input;
14 print "Your grade is: " + classify(score);
```

8.4 Shopping Cart: A Study in Language Limitations

This example is deliberately honest. It opens with the code the author *wanted* to write, then shows what CVM++ *actually* required. The gap between the two reveals the language's current boundaries in a practical, non-abstract way.

Listing 8: shopping.cvm — the ideal vs the actual

```
// What we WANTED to write:
//   let prices = [299, 149, 499, 99];
//   let total = 0.0;
//   for each price in prices { total += price * 1.18; }
//
// What we ACTUALLY had to do:

// No arrays -- items are individual variables
let item1 = 299;
let item2 = 149;
let item3 = 499;
let item4 = 99;

// No floats -- 18% tax via integer scaling: (price * 118) / 100
fn withTax(price) { return (price * 118) / 100; }

fn applyDiscount(b) {
    if (b > 900) { return (b * 90) / 100; }
    return b;
}

let bill  = withTax(item1 + item2 + item3 + item4);
let final = applyDiscount(bill);
print "Post-tax total: " + bill;
print "After discount: " + final;
let name = sinput;
print "Thank you for shopping, " + name + "!";
```

The example exposes two fundamental limitations in a natural context: the absence of arrays forces manual variable enumeration, and the absence of floating-point arithmetic forces the programmer to scale integers by 100 to simulate two decimal places. The tax calculation `(price * 118) / 100` is the integer equivalent of `price * 1.18`, and the discount `(bill * 90) / 100` stands in for `bill * 0.90`. The truncation is silent and intentional — the language simply does not have decimals.

9 Build Instructions

9.1 Prerequisites

- C++17-capable compiler: GCC 7+, Clang 5+, or MSVC 2017+.
- CMake 3.10+ (recommended), or any direct `g++` invocation.

9.2 With CMake

```
mkdir build && cd build
cmake ..
cmake --build .
./cvm ../examples/primes.cvm
```

9.3 Direct g++

On Linux/macOS:

```
g++ -std=c++17 -Isrc src/main.cpp src/lexer.cpp src/parser.cpp \
    src/compiler.cpp src/vm.cpp -o cvm
```

On **Windows PowerShell**, the backslash line continuation is not supported. Use a backtick or write the command on a single line:

```
g++ -std=c++17 -Isrc src/main.cpp src/lexer.cpp src/parser.
   cpp src/compiler.cpp src/vm.cpp -o cvm
```

10 Design Decisions and Known Limitations

10.1 Design Decisions

- **Single-pass compilation with backpatching**: the compiler walks the AST once and emits bytecode directly. Forward jumps are handled by emitting placeholder addresses and patching them once the target position is known.
- **`dynamic_cast` for AST dispatch**: the compiler uses `dynamic_cast` on the polymorphic node hierarchy rather than the Visitor pattern. This is slightly less efficient but avoids boilerplate and keeps each case self-contained.
- **Function table separate from bytecode**: compiled functions live in an `unordered_map<string, FunctionObject>` on the `Compiler` object, passed to the VM at runtime. This avoids embedding function metadata inside the flat bytecode stream.
- **Runtime type dispatch for `ADD`**: rather than requiring the programmer to use a separate concatenation operator, `ADD` checks operand types at runtime and either adds integers or concatenates strings. This makes the language feel more natural at the cost of a single runtime branch per addition.
- **Big-endian integer encoding**: `PUSH_INT` encodes 32-bit integers in big-endian order for straightforward human inspection of raw bytecode.

10.2 Known Limitations

Limitation	Impact
No floating-point	Tax, ratios require integer scaling (see shopping.cvm)
No arrays	Collections require individual variables
255-variable cap	Per function frame; sufficient for small programs
16-bit jump targets	Programs limited to 65 535 bytes of bytecode
No scoping / shadowing	All variables are flat within their function
No first-class functions	Functions cannot be passed as arguments
No closures	Functions cannot capture outer variables
No string-to-int cast	`sinput` always returns a string

11 Possible Further Extensions

Several directions remain open for future development:

- **Arrays**: requires a heap allocator and a new `ValType::ARRAY` variant pointing to heap-allocated storage. The VM would need `ARRAY_NEW`, `ARRAY_GET`, and `ARRAY_SET` opcodes.
- **First-class functions**: add `ValType::FUNCTION` to `Value` so functions can be stored in variables and passed as arguments. Requires a `CALL_VAL` opcode that pops the function from the stack rather than reading a name from the string pool.
- **Closures**: build on first-class functions by adding upvalue support — a mechanism for a function to capture variables from its enclosing scope. This is the primary direction described in *Crafting Interpreters* Chapters 25–28.
- **Floating-point type**: add `ValType::FLOAT` and a `PUSH_FLOAT` opcode. The shopping cart example would then be writeable in its natural form.
- **Disassembler**: a debug utility that pretty-prints bytecode offsets, opcodes, and operands from a compiled `Chunk`, useful for tracing compilation bugs.
- **`break` and `continue`**: require tracking the patch position of the nearest enclosing loop exit in the compiler.

12 Conclusion

CVM++ began as a minimal stack machine that could evaluate integer arithmetic and branch on boolean conditions. Through a series of focused, incremental extensions — each following the same four-layer pattern of lexer, parser, compiler, and VM — it grew into a language capable of user-defined functions, recursion, strings, a full operator set, and three distinct loop constructs.

The project demonstrates that a layered compiler architecture is not just a theoretical ideal: every single feature added here, from the modulo operator to the

call stack, touched exactly the layers it needed to touch and nothing else. That separation is what made the project extensible in practice, not just in principle.

The language still has meaningful limitations — no arrays, no floats, no first-class functions — and the shopping cart example makes no attempt to hide them. A language that understands its own boundaries is more useful than one that pretends they do not exist.

Reference: "Crafting Interpreters" by Robert Nystrom — the natural next step for extending CVM++ with closures, garbage collection, and beyond.